CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
An Autonomous Institute I Affiliated to Osmania University
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cbit.ac.in

COMMITTED TO RESEARCH, INNOVATION AND EDUCATION 48 years

C O U R S E - E N D P R O J E C T R E P O R T
O n

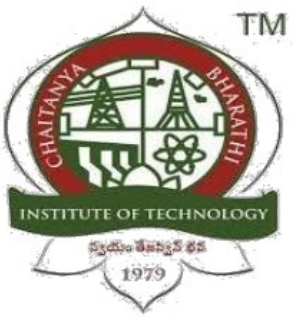
Exploratory Data Analysis and Train Passenger Travel Pattern Analysis

Submitted to fulfill the requirements for the Award of CIE Marks (10) for the B.E.(IT), III-SEM, Course:
Exploratory Data Analysis & Visualization - EDAV (22ADC32N)

By

RAKESH NALLA
160125737112

Under the guidance of
Mr. G. Srikanth
Assistant Professor, Dept. of IT, CBIT



CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (CBIT)
DEPARTMENT OF INFORMATION TECHNOLOGY (IT)
KOKAPET (V), GANDIPETA, RANGA REDDY, HYDERABAD – 500075
www.cbit.ac.in

A.Y: 2026-27

CERTIFICATE

This is to certify that the course-end project report entitled **"Exploratory Data Analysis and Train Passenger Travel Pattern Analysis"** is a bonafide record of the work carried out by **RAKESH NALLA (160125737112)**, in partial fulfilment of the requirements of the course **Exploratory Data Analysis and Visualization (22ADC32N)** of the programme **B.E(IT), III-SEM**, during the academic year **2026–2027**.

The project was carried out under my guidance and supervision. To the best of my knowledge, the work reported here is original and has not been submitted to any other institution or university for the award of any degree, diploma or certificate.

Place: Hyderabad
Date: ___/___/_____

Course Faculty
(Mr. G. Srikanth)
Assistant Professor, Dept. of IT, CBIT

DECLARATION

I hereby declare that the project report entitled **"Exploratory Data Analysis and Train Passenger Travel Pattern Analysis"** submitted to **Dept of IT, CBIT** is a record of original work done by me under the guidance of **Mr. G. Srikanth, Assistant Professor, IT, CBIT**.

The data used was taken from the Train Ticket Sales India dataset (train_ticket_sales_india.csv) used for the course; all external sources, tools and libraries have been acknowledged in the References. This work has not been submitted, in part or in full, for any other course, degree or diploma.

Place: Hyderabad
Date: ___/___/_____

Signature of the Student
RAKESH NALLA (160125737112)

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide and course faculty, Mr. G. Srikanth, Assistant Professor, Dept. of IT, CBIT, for valuable guidance, encouragement and constructive feedback throughout this project.

I thank the Head of the Department, the faculty members and the management of Chaitanya Bharathi Institute of Technology (CBIT) for providing the facilities and the learning environment that made this work possible.

I am also grateful to the open-source community behind Python, pandas, NumPy, Matplotlib and Seaborn, and to the publishers of the Train Ticket Sales India dataset, without which this analysis would not have been possible. Finally, I thank my family and friends for their constant support.

Rakesh Nalla

Contents

S.No.	Item	Page No
	Abstract	6
1	Introduction	7
2	Problem Statement and Objectives	7
3	Dataset Description	8
4	Methodology and Tools	8
5	Data Quality Assessment and Cleaning	9
6	Exploratory Data Analysis	10
7	Visualization and Distribution Analysis	16
8	Station and Route Demand Analysis	19
9	Key Findings	21
10	Recommendations	22
11	Limitations	22
12	Conclusion	23
13	Future Scope	23
	References	24
	Appendix A – Notebook (ipynb) File	25
	Appendix B – How to Run	32

Abstract

Rail travel is the backbone of passenger movement in India, and the tickets sold by a railway network record who travels, between which stations, in which coach class and at what price. This project analyses the Train Ticket Sales India dataset of 8,000 ticket records sold between January 2023 and September 2025, of which 6,738 valid records remain after cleaning, to understand how passenger demand, ticket price and payment behaviour vary with route, coach type, passenger age and time.

The work follows an end-to-end data analysis workflow in Python: dataset loading, data-quality assessment and cleaning (harmonising station codes and labels, removing journeys whose source and destination are the same), feature engineering (route, age group, price level, payment code and price rank), Pandas indexing, merging, pivoting and group-by analysis, time-series resampling, NumPy statistical analysis, and visualization with Matplotlib and Seaborn (line, bar, histogram, scatter, box and violin plots, a route heat map and a pair plot).

Main results. Sleeper is the most booked coach (3,380 tickets, 50.2%), followed by General (1,965; 29.2%) and AC (1,393; 20.7%). Ticket price is determined mainly by coach class: the mean price is Rs. 2,016.30 for AC, Rs. 897.84 for Sleeper and Rs. 349.82 for General, with maximum prices of Rs. 2,500, Rs. 1,200 and Rs. 500. AC accounts for 20.7% of tickets but about 43.0% of ticket revenue. The overall mean ticket price is Rs. 969.25 (median Rs. 844). Demand is concentrated on the Delhi, Mumbai and Chennai corridors: each of the six directed routes between these three cities carries between 314 and 337 tickets, and the three cities together are the source of 57.6% of all tickets. Passenger age is spread almost evenly between about 5 and 80 years and shows no visible relation with price. Monthly ticket revenue fluctuates between roughly Rs. 170,000 and Rs. 215,000 without a clear long-term trend.

Keywords: railway passenger analysis; exploratory data analysis; ticket sales; route demand; pandas; NumPy; Matplotlib; Seaborn.

1. Introduction

Railways are the most widely used long-distance mode of travel in India. Every ticket sold records a small piece of travel behaviour: the day it was bought, the two stations it connects, the age of the passenger, the class of coach and the price paid. Taken together, thousands of tickets show which routes are busy, which coach classes passengers prefer and how revenue moves through the year. These are practical questions for railway planners, who must decide how much capacity to provide, how to mix coach classes and where to improve passenger services.

The available data records, for every ticket, a ticket ID, the sale date, the source and destination station, the passenger age, the coach type, the ticket price and the payment mode. Such data is well suited to a teaching project because it needs every stage of a real analysis: loading, inspecting, cleaning, transforming, indexing and merging, grouping, resampling over time, visualizing and drawing conclusions. Its imperfections mirror those of real transaction data: the same station is recorded both as a city name and as a station code (for example BCT, MAS, NDLS, HWH and SBC), the payment label NetBanking is written without a space, and some tickets have the same source and destination.

This report describes the analysis carried out in the companion Jupyter notebook (Appendix A). Every table and figure in the report is taken from, or computed directly from, the outputs of that notebook, so the results can be reproduced. Where the notebook does not contain a result (for example the number of tickets per payment mode), the report says so instead of estimating it.

2. Problem Statement and Objectives

Problem statement. To analyse the Train Ticket Sales India dataset in order to describe how ticket demand, ticket price and payment behaviour vary across routes, stations, coach types, passenger age groups and time, and to identify the travel patterns that can support railway planning and passenger services.

2.1 Objectives

1. Load the dataset and examine its structure, data types and quality.
2. Clean the data by harmonising station codes and labels, converting data types, checking for missing values and duplicates and removing journeys whose source and destination are identical.
3. Create derived features: route, age group, price level, price rank and a numeric payment code.
4. Summarise tickets and prices with Pandas selection, indexing, merging, pivoting and group-by analysis.
5. Analyse ticket revenue over time by resampling the sale dates.
6. Compute statistical summaries and array operations with NumPy.
7. Create Matplotlib and Seaborn visualizations (line, bar, histogram, scatter, box, violin, heat map and pair plot) and interpret them.
8. Identify the busiest routes and stations and derive findings and recommendations.

2.2 Research questions

- How are tickets distributed over coach types, and how much revenue does each coach type contribute?
- How does the ticket price vary with coach type, source station, payment mode and passenger age?
- Which routes and stations carry the most passengers?
- What is the age profile of the passengers?

- How does monthly ticket revenue change between January 2023 and September 2025?

3. Dataset Description

The analysis uses the file train_ticket_sales_india.csv, which is uploaded into Google Colab at the start of the notebook. According to the notebook, the file holds 8,000 ticket records sold between January 2023 and September 2025 in 8 columns. After cleaning, 6,738 tickets remain; they connect 7 cities (Bengaluru, Chennai, Delhi, Gorakhpur, Jaipur, Kolkata and Mumbai), use 3 coach types (General, Sleeper and AC) and are paid for in 4 ways (Cash, UPI, Card and Net Banking). Each row describes one ticket. The currency of the ticket price is not stated in the file; Indian Rupees (Rs.) are assumed in this report. The original source of the file is not stated in the notebook (the cell contains the placeholder “add dataset link here”) and should be cited by the student in the final submission.

Table 1: Columns of the dataset (train_ticket_sales_india.csv)

Column	Type	Description
Ticket_ID	Number	Serial number of the ticket record
Sale_Date	Date	Date on which the ticket was sold (converted from text to datetime during cleaning)
Source_Station	Text	Station where the journey starts; city name or station code before cleaning
Destination_Station	Text	Station where the journey ends; city name or station code before cleaning
Passenger_Age	Number	Age of the passenger in years
Coach_Type	Text	Class of coach: General, Sleeper or AC
Ticket_Price	Number	Price of the ticket (converted to a decimal number during cleaning)
Payment_Mode	Text	Mode of payment: Cash, UPI, Card or Net Banking (written NetBanking in the raw file)

The data contains no train number, journey date or time, travel distance or seat information. The analysis is therefore restricted to what the eight columns can show: demand by route and station, price by coach class, the age profile of passengers, payment behaviour and ticket revenue by sale date.

4. Methodology and Tools

The project follows a seven-step pipeline: (1) set up the environment and load the dataset, (2) inspect the data, (3) clean it, (4) derive new features, (5) index, merge, reshape and group the data, (6) analyse it over time and with NumPy and visualize the results, and (7) synthesise the findings. Every step is implemented in the companion notebook (Appendix A), so the results are reproducible. The notebook sections are listed in Table 3.

Table 2: Libraries used

Purpose	Library
Numerical statistics and arrays	NumPy (mean, median, standard deviation, arrays, broadcasting, matrix product, random numbers)
Data handling, group-by and time series	pandas (read_csv, replace, drop_duplicates, cut, qcut, merge, pivot, groupby, resample)
Basic charts	Matplotlib (subplots, line plot, bar chart, histogram, scatter plot)
Statistical charts	Seaborn (box plot, violin plot, heat map, pair plot)
File upload in Google Colab	google.colab.files (upload of the CSV file)

Table 3: Notebook sections and main techniques

Section	Task	Main techniques
Setup and loading	Import libraries, upload and read the CSV file	read_csv, files.upload
Pandas basics	Series and DataFrame creation, selection of rows and columns	Series, DataFrame, loc, iloc, Boolean filter
Cleaning	Missing values, station codes, labels, data types, duplicates, invalid journeys	isnull, fillna, dropna, replace, to_datetime, drop_duplicates
Transformation	Derived columns, sorting and querying	map, cut, qcut, rank, sort_values, query
Indexing and merging	Hierarchical index, concatenation, merge, reshaping	set_index, concat, merge, pivot, stack, unstack
Group-by and time series	Group summaries, monthly and weekly resampling	groupby, agg, resample, date slicing
Matplotlib	Four-panel figure	subplots, plot, hist, scatter
Seaborn	Box, violin, heat map and pair plot	boxplot, violinplot, heatmap, pairplot
NumPy	Arrays, indexing, broadcasting, statistics	arange, reshape, mean, median, std

Key technical choices: station codes are replaced by city names with a dictionary so that each city has one label; journeys whose source and destination are the same are removed as invalid; ticket prices are converted to decimal numbers and sale dates to datetime values so that the data can be resampled by month; passengers are grouped into four age groups with pd.cut and ticket prices into three levels with pd.qcut; and monthly ticket revenue is obtained by summing ticket prices over calendar months.

5. Data Quality Assessment and Cleaning

The data was checked before analysis. The issues found and the treatment adopted are summarised in Table 4. The notebook did not change genuine observations; it only harmonised labels, converted data types and removed invalid journeys.

Table 4: Data-quality issues and treatment

Issue	Evidence	Treatment
Missing values	isnull().sum() is 0 for all 8 columns	Median fill for age and dropna on price were applied as safeguards; no value changed
Duplicate rows	duplicated().sum() is 0	drop_duplicates applied; no row removed
Station recorded as code	Codes BCT, MAS, NDLS, HWH and SBC appear beside city names	Replaced by Mumbai, Chennai, Delhi, Kolkata and Bengaluru
Inconsistent payment label	NetBanking written without a space	Replaced by Net Banking
Wrong data types	Sale_Date is text; Ticket_Price is an integer	Converted to datetime and to float
Same source and destination	Rows such as Delhi to Delhi (ticket 5)	Removed; 6,738 of 8,000 rows remain (1,262 removed)

After cleaning, five columns were added by the notebook: Pay_Code, Route, Age_Group, Price_Level and Price_Rank (Table 5). Column names of the original file were kept. The age groups use right-inclusive bins, so an age of exactly 18 falls in the Child group.

Table 5: Derived columns

Column	Definition	Values
Pay_Code	Numeric code of the payment mode (map)	Cash 0, UPI 1, Card 2, Net Banking 3
Route	Source_Station + " - " + Destination_Station	For example Bengaluru - Delhi
Age_Group	pd.cut of Passenger_Age with bins 0, 18, 35, 60, 100	Child, Young, Adult, Senior
Price_Level	pd.qcut of Ticket_Price into three equal-sized groups	Low, Medium, High
Price_Rank	Rank of the ticket price in descending order; ties receive the average rank	Number from 1 to 6,738

The first five raw records (Table 6) show why cleaning was needed: stations appear as codes (SBC, BCT, HWH) and ticket 5 starts and ends in Delhi. Table 7 shows the first three cleaned records with the two derived columns Pay_Code and Route.

Table 6: First five raw records (before cleaning)

ID	Sale date	Source	Destination	Age	Coach	Price (Rs.)	Payment
1	2023-01-01	SBC	Delhi	24	Sleeper	1,065	Cash
2	2023-01-01	Mumbai	Chennai	46	General	338	Cash
3	2023-01-01	SBC	Mumbai	76	General	420	UPI
4	2023-01-01	BCT	HWH	38	Sleeper	872	UPI
5	2023-01-01	Delhi	Delhi	54	AC	1,653	UPI

Table 7: First three cleaned records with derived columns

ID	Sale date	Source	Destination	Age	Coach	Price (Rs.)	Payment	Pay_Code	Route
1	2023-01-01	Bengaluru	Delhi	24	Sleeper	1,065	Cash	0	Bengaluru - Delhi
2	2023-01-01	Mumbai	Chennai	46	General	338	Cash	0	Mumbai - Chennai
3	2023-01-01	Bengaluru	Mumbai	76	General	420	UPI	1	Bengaluru - Mumbai

The effect of the cleaning steps on the size of the data is summarised in Table 8. Since no duplicates and no missing values were found, all 1,262 removed rows are the journeys with identical source and destination, which is 15.8% of the raw records. The row counts are taken from the notebook: 8,000 raw records (stated in the first cell) and 6,738 after cleaning (output of the cleaning cell).

Table 8: Effect of cleaning on the dataset

Stage	Rows	Columns	Missing values
Raw file	8,000	8	0
After cleaning (duplicates, same-station journeys)	6,738	8	0
After adding the derived columns	6,738	13	0

One limitation of the duplicate check should be noted: because every record carries a unique Ticket_ID, drop_duplicates compares whole rows including the ID and can never find a repeated booking. Duplicates were therefore not detected in a meaningful sense.

6. Exploratory Data Analysis

6.1 Pandas basics and data selection

The notebook first builds a Series from the first five ticket prices with the labels t1 to t5, and a small look-up DataFrame that assigns a numeric level to every coach type (General 1, Sleeper 2, AC 3). Both are shown below; the look-up table is used again in the merge operation of Section 6.3.

Table 9: Pandas Series of the first five ticket prices

Label	t1	t2	t3	t4	t5
Ticket price (Rs.)	1,065	338	420	872	1,653

Table 10: Coach-type look-up DataFrame

Coach_Type	Level
General	1
Sleeper	2
AC	3

Label-based selection with loc and position-based selection with iloc return the source, destination and price of the first five tickets, and the first four columns of the first five rows. A Boolean filter then keeps the tickets priced above the mean. Table 11 shows the first five of them; all are Sleeper or AC tickets, which is expected because General tickets never reach the mean price (Section 6.4). These outputs were produced before the cleaning step, so station codes still appear.

Table 11: First five tickets priced above the mean (before cleaning)

ID	Source	Destination	Age	Coach	Price (Rs.)	Payment
1	SBC	Delhi	24	Sleeper	1,065	Cash
5	Delhi	Delhi	54	AC	1,653	UPI
6	Mumbai	SBC	15	AC	1,748	Cash
7	HWH	Gorakhpur	47	AC	1,778	UPI
9	MAS	Mumbai	12	Sleeper	1,113	NetBanking

6.2 Data transformation and feature engineering

Sorting the cleaned data by ticket price in descending order gives the five most expensive tickets (Table 12). All five are AC tickets sold to passengers aged between 14 and 61 years, on three different routes from or to Gorakhpur, Delhi and Chennai. Three tickets share the maximum price of Rs. 2,500, and the next group of tied tickets at Rs. 2,499 has an average rank of 5.0, which shows that tied prices receive the mean of their ranks.

Table 12: Five highest ticket prices (sorted with sort_values)

ID	Sale date	Route	Age	Coach	Price (Rs.)	Payment	Age group	Rank
7591	2025-08-04	Gorakhpur - Delhi	32	AC	2,500	Cash	Young	2.0
4746	2024-08-15	Gorakhpur - Jaipur	27	AC	2,500	Net Banking	Young	2.0
2422	2023-10-30	Delhi - Jaipur	14	AC	2,500	Cash	Child	2.0
7802	2025-08-31	Delhi - Mumbai	61	AC	2,499	Cash	Senior	5.0
2119	2023-09-22	Chennai - Gorakhpur	31	AC	2,499	Cash	Young	5.0

The query Age_Group == “Senior” and Price_Level == “High” selects senior passengers who bought high-priced tickets (Table 13). The result contains both AC tickets and Sleeper tickets priced above Rs. 1,000, so the upper price tercile includes the upper part of the Sleeper range as well as all AC tickets; the boundary of the High level therefore lies below Rs. 1,055.

Table 13: Senior passengers with high-priced tickets (first five results of query)

ID	Sale date	Route	Age	Coach	Price (Rs.)	Payment
32	2023-01-04	Kolkata - Jaipur	71	AC	1,860	UPI
37	2023-01-05	Mumbai - Delhi	69	Sleeper	1,055	Net Banking
39	2023-01-05	Mumbai - Chennai	68	Sleeper	1,065	UPI
43	2023-01-06	Jaipur - Chennai	77	AC	1,972	UPI
47	2023-01-06	Gorakhpur - Mumbai	61	Sleeper	1,121	Card

6.3 Indexing, merging and reshaping

Setting Source_Station and Destination_Station as a hierarchical (MultiIndex) index and sorting it allows all tickets of one source city to be selected with a single call. multi.loc[“Delhi”] returns the tickets that start in Delhi, grouped by destination; the first rows are Delhi to Bengaluru tickets (Table 14).

Table 14: First rows of multi.loc["Delhi"] (Delhi to Bengaluru tickets)

Ticket_ID	Sale date	Age	Coach	Price (Rs.)	Payment	Age group	Price level
12	2023-01-02	69	General	329	Cash	Senior	Low
56	2023-01-07	68	General	232	UPI	Senior	Low
100	2023-01-13	75	AC	2,197	Cash	Senior	High
137	2023-01-17	68	Sleeper	714	Card	Senior	Medium
278	2023-02-04	34	Sleeper	916	Cash	Young	Medium

Concatenating the first three and the last three rows with pd.concat gives a table of 6 rows and 13 columns. Merging the data with the coach-level look-up table on Coach_Type (a left join) keeps all 6,738 rows and adds one column, Level, so the merged table has 6,738 rows and 14 columns; the first five levels are 2, 1, 1, 2 and 3 (Sleeper, General, General, Sleeper and AC). No rows were duplicated or lost by the merge.

The mean ticket price of every source station and coach type was computed with groupby and reshaped into a table with pivot (Table 15). pv.stack() turns this table back into a long series with one value per station and coach type, and unstack() restores the wide form. Figure 1 shows the same values as grouped bars.

Table 15: Mean ticket price by source station and coach type (Rs., rounded)

Source station	AC	General	Sleeper
Bengaluru	2,005	346	896
Chennai	2,052	346	909
Delhi	2,013	349	899
Gorakhpur	1,998	352	894
Jaipur	2,014	357	894
Kolkata	2,014	349	899
Mumbai	2,003	352	891

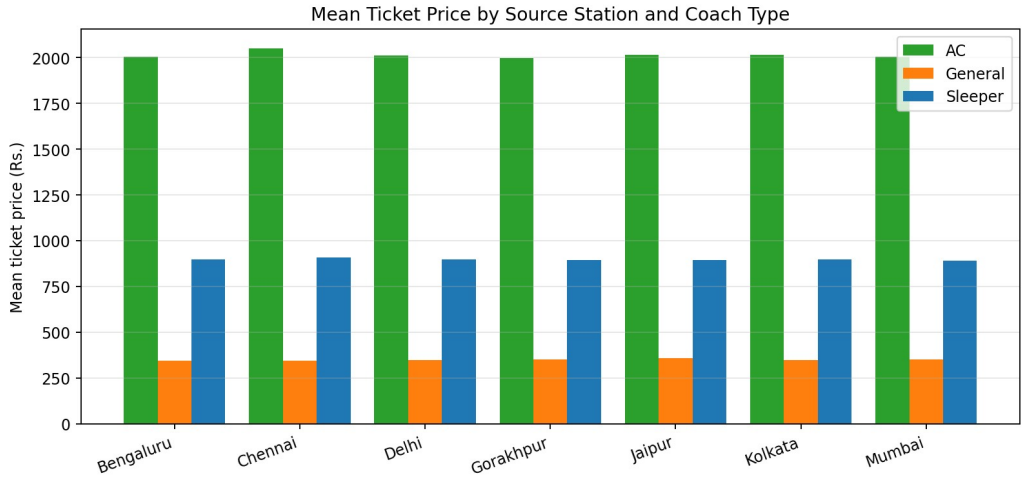


Figure 1: *Mean ticket price by source station and coach type.*

The price of a ticket hardly depends on the station where the journey starts: the mean AC price lies between Rs. 1,998 and Rs. 2,052, the mean Sleeper price between Rs. 891 and Rs. 909 and the mean General price between Rs. 346 and Rs. 357. Chennai has the highest mean AC and Sleeper prices, but the differences between stations are small compared with the differences between coach types.

6.4 Group-by analysis

Grouping the tickets by coach type gives the count, the mean price and the maximum price of each class (Table 16). Sleeper is the most frequently booked coach with 3,380 tickets, followed by General (1,965) and AC (1,393). The mean price rises from Rs. 349.82 for General to Rs. 897.84 for Sleeper and Rs. 2,016.30 for AC, and the maximum price of each class (Rs. 500, Rs. 1,200 and Rs. 2,500) is far from the minimum of the next class, so the three price ranges do not overlap.

Table 16: Tickets and prices by coach type

Coach type	Tickets	Mean price (Rs.)	Maximum price (Rs.)
AC	1,393	2,016.30	2,500
General	1,965	349.82	500
Sleeper	3,380	897.84	1,200

Multiplying the number of tickets by the mean price of each class gives the ticket revenue of the class (Table 17). These figures are derived from the notebook output; their sum, Rs. 6,530,806, equals the overall mean price (Rs. 969.25) multiplied by the 6,738 tickets, which confirms the calculation. AC contributes 43.0% of the revenue with only 20.7% of the tickets, whereas General contributes only 10.5% of the revenue with 29.2% of the tickets (Figure 2).

Table 17: Share of tickets and ticket revenue by coach type (derived)

Coach type	Tickets	Share of tickets	Revenue (Rs.)	Share of revenue
Sleeper	3,380	50.2%	3,034,700	46.5%
General	1,965	29.2%	687,398	10.5%
AC	1,393	20.7%	2,808,708	43.0%
Total	6,738	100.0%	6,530,806	100.0%

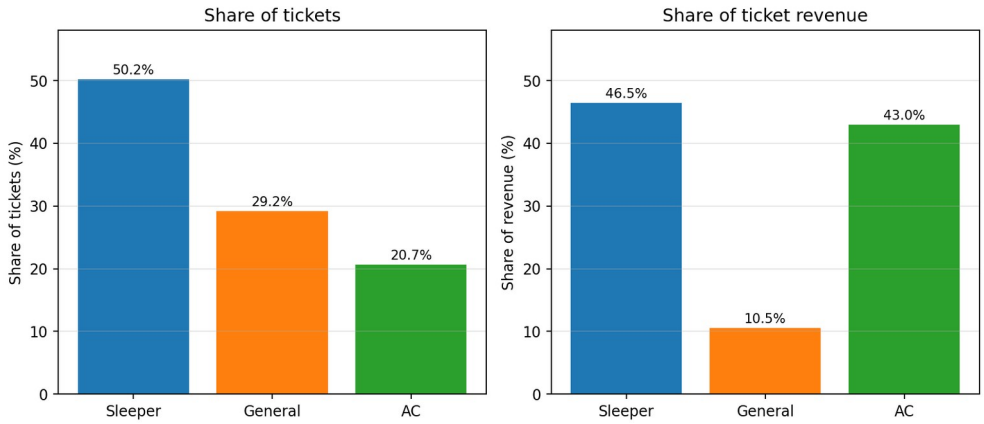


Figure 2: *Share of tickets and share of ticket revenue by coach type.*

The notebook also computes the mean ticket price for every combination of payment mode and age group (Table 18 and Figure 3). The values lie between Rs. 899.17 (Cash, Adult) and Rs. 1,024.54 (Net Banking, Young), within about 7% of the overall mean of Rs. 969.25 on either side, and no payment mode is consistently cheaper or dearer than another. The tickets per payment mode and per age group are not counted in the notebook, so a statement about how many passengers use each payment mode cannot be made from the notebook outputs.

Table 18: Mean ticket price (Rs.) by payment mode and age group

Payment mode	Child	Young	Adult	Senior
Card	1,015.25	946.65	1,011.51	978.21
Cash	956.17	1,003.05	899.17	965.97
Net Banking	963.49	1,024.54	943.84	958.69
UPI	922.06	970.36	987.61	969.41

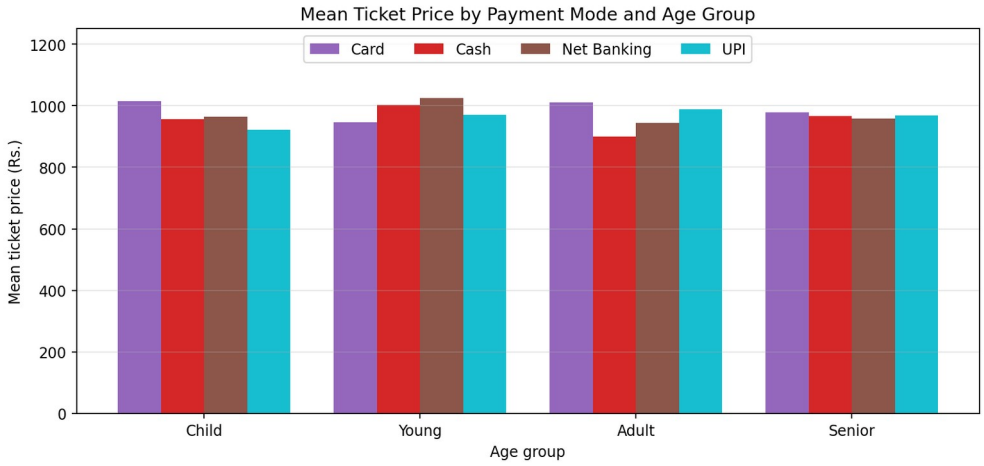


Figure 3: *Mean ticket price by payment mode and age group.*

6.5 Time-series analysis

For the time-series analysis the sale date is set as the index and the data is sorted by date. Ticket prices are then summed by calendar month with resample("ME") to give the monthly ticket revenue, and averaged by week with resample("W") to give the weekly mean price; the weekly series is computed in the notebook but is not displayed or plotted. Table 19 shows the monthly revenue of the first five months. It stays between Rs. 196,739 and Rs. 208,642, with the highest value in January 2023 and the lowest in May 2023. The complete series is plotted in Figure 4 (Section 7.1).

Table 19: Monthly ticket revenue, January to May 2023 (resample sum)

Month	Jan 2023	Feb 2023	Mar 2023	Apr 2023	May 2023
Ticket revenue (Rs.)	208,642	198,804	207,156	204,980	196,739

Because the date is the index, a slice such as ts.loc["2024-01":"2024-03"] returns all tickets of the first quarter of 2024. The first five rows of this slice are the tickets sold on 1 January 2024 (Table 20), which have the Ticket_ID values 2927 to 2931.

Table 20: First five rows of the slice 2024-01 to 2024-03

Sale date	ID	Route	Age	Coach	Price (Rs.)	Payment	Age group
2024-01-01	2927	Mumbai - Jaipur	20	General	437	Card	Young
2024-01-01	2928	Chennai - Jaipur	17	General	402	UPI	Child
2024-01-01	2929	Delhi - Mumbai	15	General	446	Net Banking	Child
2024-01-01	2930	Chennai - Mumbai	6	Sleeper	1,192	Card	Child
2024-01-01	2931	Jaipur - Delhi	23	Sleeper	955	Net Banking	Young

6.6 NumPy statistical analysis

The ticket prices of the cleaned data are converted into a NumPy array and the notebook demonstrates the main array operations on small arrays: a one-dimensional array of the numbers 1 to 9, the same numbers reshaped into a 3 × 3 matrix, and a 2 × 3 × 4 array of the numbers 0 to 23. The operations and their outputs are listed in Table 21. These operations are general demonstrations of NumPy and do not use the ticket data.

Table 21: NumPy array operations and their results

Operation	Result
a2[1, :] (second row of the 3 × 3 matrix)	[4 5 6]
a2[:2, 1:] (first two rows, last two columns)	[[2 3] [5 6]]
a3[1, 2, 3] (element of the 3-D array)	23
a2 + [10, 20, 30] (broadcasting)	[[11 22 33] [14 25 36] [17 28 39]]
a2 @ a2.T (matrix product)	[[14 32 50] [32 77 122] [50 122 194]]
a2.sum(axis=0) (column sums)	[12 15 18]
a2.mean(axis=1) (row means)	[2. 5. 8.]

Applied to the ticket prices, NumPy gives the summary in Table 22. The mean price (Rs. 969.25) is higher than the median (Rs. 844), which indicates a right-skewed distribution: the AC tickets, with prices above Rs. 1,500, pull the mean upwards. The standard deviation of Rs. 613.41 is large because the price is spread over three separate classes. The maximum price of Rs. 2,500 lies about 2.5 standard deviations above the mean. The notebook also standardises the prices into z-scores; the first three tickets (Rs. 1,065, Rs. 338 and Rs. 420) have z-scores of 0.156, −1.029 and −0.895. A random sample of 50 prices, five random integers and three random numbers are drawn with a fixed seed of 1 to demonstrate NumPy random functions.

Table 22: NumPy statistics of the ticket price (6,738 tickets)

Measure	Mean	Median	Std dev	Max
Ticket price (Rs.)	969.25	844.00	613.41	2,500.00

The standard deviation is the population value (np.std with ddof = 0). The notebook does not print the minimum price, the quartiles or the output of df.describe(), because that cell stopped with an error (Section 11).

7. Visualization and Distribution Analysis

7.1 Matplotlib charts: revenue, coach type, age and price

The first set of charts was produced with Matplotlib in a 2 × 2 layout; they are shown here as four separate figures. Figure 4 plots the monthly ticket revenue from January 2023 to September 2025 with one marker per month. The revenue moves between about Rs. 170,000 and Rs. 215,000 without a sustained rise or fall. There are several isolated drops to roughly Rs. 180,000, and the series ends with its lowest point (about Rs. 169,000) and its highest point (about Rs. 214,500) in consecutive months at the right edge of the chart. The notebook does not show whether the last month of the data is complete, so the final drop should be treated with caution. All readings from the chart are approximate; the exact monthly values are not printed in the notebook beyond the first five months. Note that the chart title says ticket sales, but the plotted quantity is the sum of ticket prices, that is, revenue, not the number of tickets.

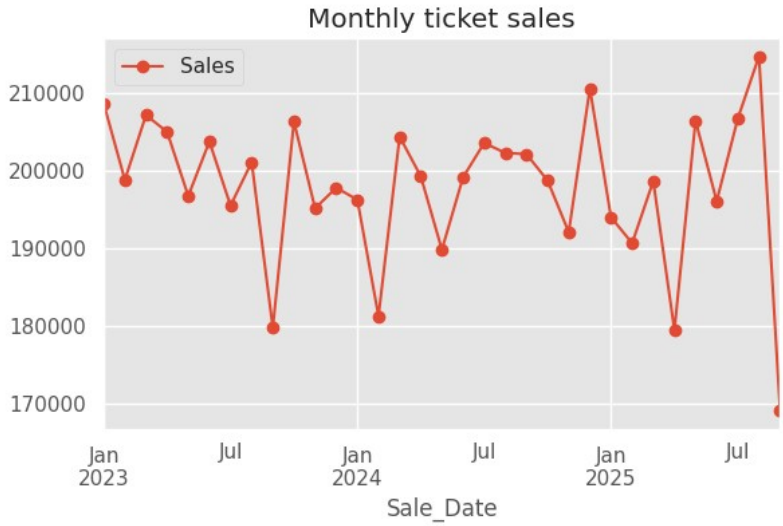


Figure 4: *Monthly ticket revenue (sum of ticket prices), January 2023 to September 2025.*

Figure 5 confirms the group-by result: Sleeper is the most booked coach type, followed by General and AC.

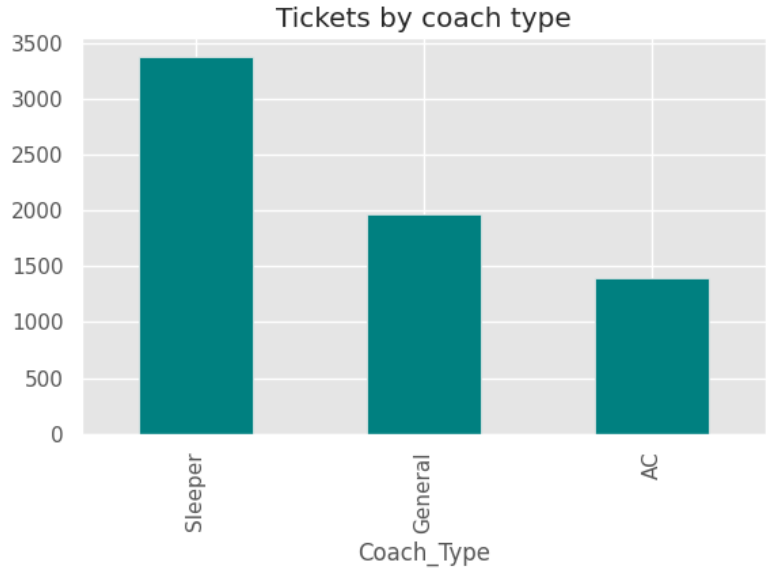


Figure 5: *Number of tickets by coach type.*

The age histogram in Figure 6 has 15 bins covering the ages from about 5 to 80 years. Every bin holds roughly 415 to 540 passengers, and the last bin (the oldest passengers) is the highest. The

distribution is therefore close to uniform: no age group dominates and passengers of every age buy tickets.

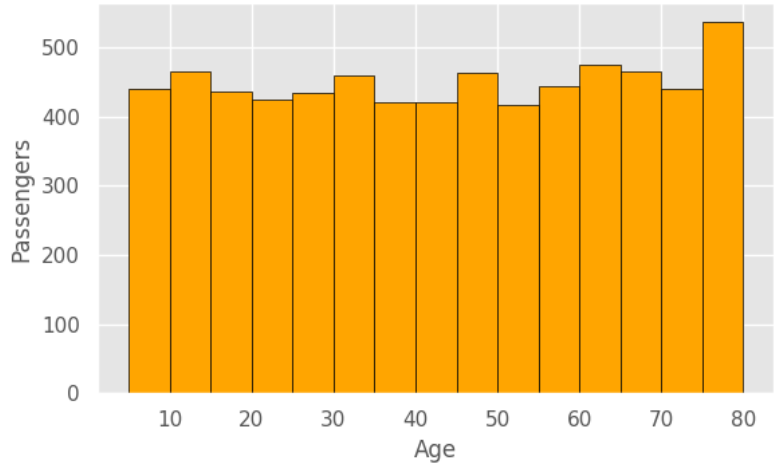


Figure 6: *Distribution of passenger age.*

In the scatter plot of age against ticket price (Figure 7) the points form three horizontal bands: about Rs. 200–500 (General), Rs. 600–1,200 (Sleeper) and Rs. 1,500–2,500 (AC), with empty gaps between them. Each band extends over the whole age range, so passengers of every age pay similar prices within a coach type. The price is a function of the coach class and not of the age of the passenger.

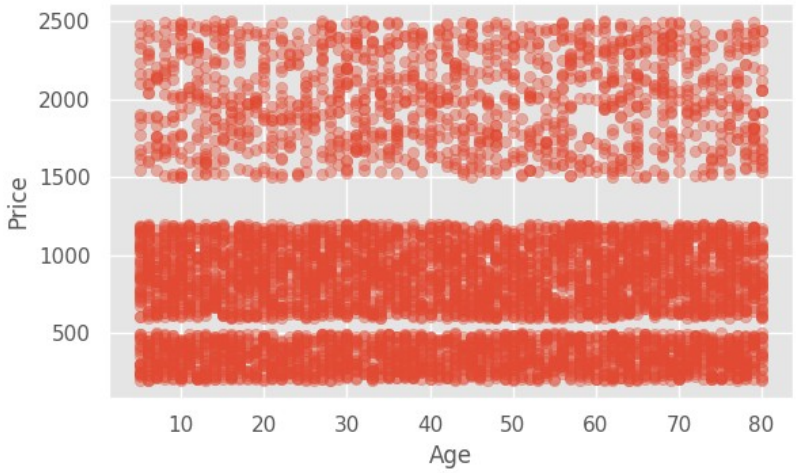


Figure 7: *Passenger age versus ticket price.*

7.2 Seaborn charts: price and age distributions

The box plot in Figure 8 compares the price distribution of the three coach types. The AC box lies between about Rs. 1,770 and Rs. 2,290 with a median near Rs. 2,000; the Sleeper box lies between about Rs. 740 and Rs. 1,050 with a median near Rs. 890; the General box lies between about Rs. 270 and Rs. 420 with a median near Rs. 350. The three boxes do not overlap, and no outliers are drawn. These values are read from the chart and are approximate.

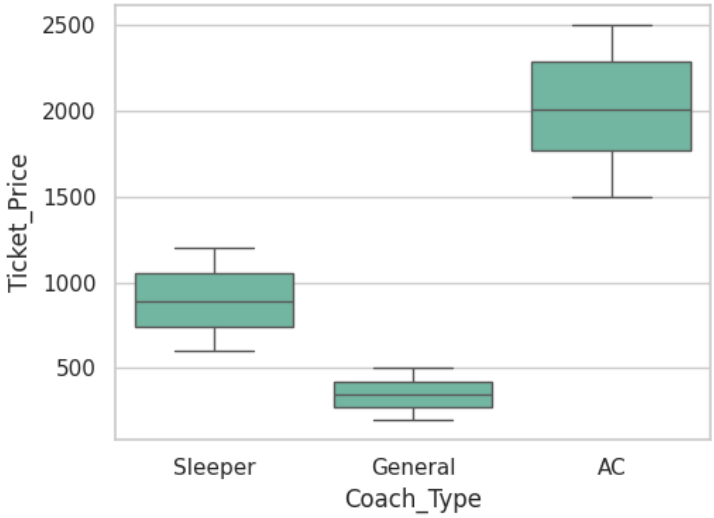


Figure 8: *Ticket price by coach type (box plot).*

The violin plot in Figure 9 compares the age distribution for each payment mode. The four violins have almost the same shape, spanning the ages from about 5 to 80 years with a median age of about 42 to 44 years. The age profile of passengers is therefore similar for Cash, UPI, Net Banking and Card payments.

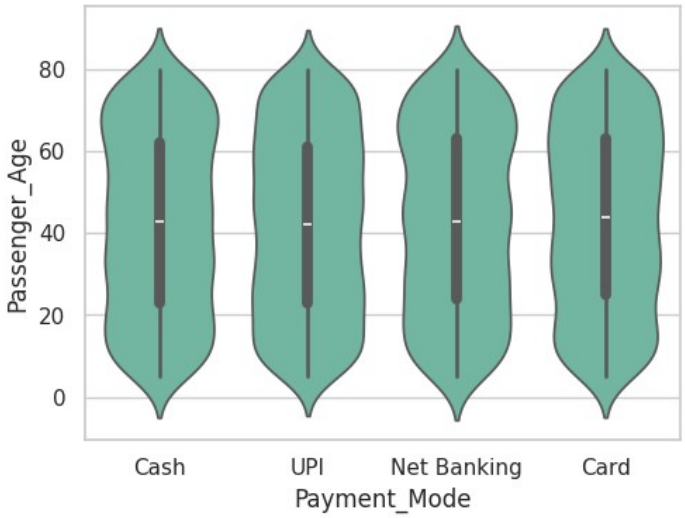


Figure 9: *Passenger age by payment mode (violin plot).*

7.3 Route heat map

A pivot table counts the tickets for every pair of source and destination station, and sns.heatmap draws it as a colour-coded matrix (Figure 10). The rows are the source stations and the columns the destination stations; the diagonal is empty because journeys with the same source and destination were removed. The darkest cells are the routes between Chennai, Delhi and Mumbai (314 to 337 tickets); the lightest cells are routes between the smaller stations Bengaluru, Gorakhpur, Jaipur and Kolkata (77 to 98 tickets). Section 8 analyses the route counts in detail.

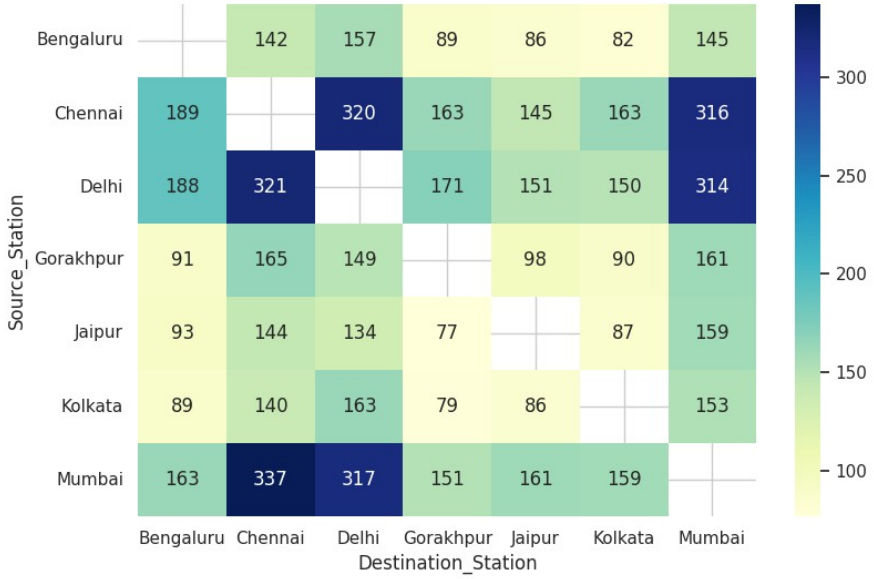


Figure 10: *Heat map of the number of tickets by source and destination station.*

7.4 Pair plot

The pair plot in Figure 11 shows passenger age and ticket price for the three coach types. On the diagonal the age densities of the three coach types are similar and almost flat between 5 and 80 years, while the price densities form three separate clusters: a narrow one near Rs. 350 for General, a broad one between Rs. 600 and Rs. 1,200 for Sleeper and a broad one between Rs. 1,500 and Rs. 2,500 for AC. The scatter panels again show that the price bands are independent of age. The pair plot therefore agrees with the box plot and the scatter plot.

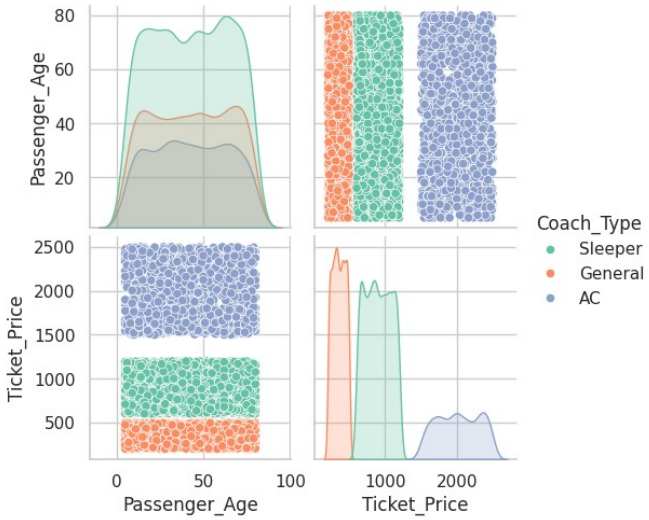


Figure 11: *Pair plot of passenger age and ticket price by coach type.*

8. Station and Route Demand Analysis

As an additional analysis, the route counts of the heat map (Figure 10) were tabulated and summed by station. The 42 directed route counts were read from the notebook heat map and add up to exactly 6,738 tickets, the number of cleaned records, which confirms that they were read correctly. Table 23 gives the complete matrix with the row and column totals.

Table 23: Number of tickets by source (rows) and destination (columns) station

Source \ Destination	Beng.	Chen.	Delhi	Gora.	Jaipur	Kolk.	Mum.	Total
Bengaluru	–	142	157	89	86	82	145	701
Chennai	189	–	320	163	145	163	316	1,296
Delhi	188	321	–	171	151	150	314	1,295
Gorakhpur	91	165	149	–	98	90	161	754
Jaipur	93	144	134	77	–	87	159	694
Kolkata	89	140	163	79	86	–	153	710
Mumbai	163	337	317	151	161	159	–	1,288
Total	813	1,249	1,240	730	727	731	1,248	6,738

Each of the 42 directed routes carries between 77 tickets (Jaipur to Gorakhpur) and 337 tickets (Mumbai to Chennai), with an average of 160.4 tickets per route. Demand is clearly concentrated. The six directed routes among Chennai, Delhi and Mumbai carry 314 to 337 tickets each, together 1,925 tickets or 28.6% of all tickets, although they are only 6 of the 42 routes (14.3%). Table 24 and Figure 12 show the same concentration by station: Chennai, Delhi and Mumbai are each the source of about 1,290 tickets and together of 3,879 tickets (57.6% of all tickets), and the destination of 3,737 tickets (55.5%).

Table 24: Tickets by source and destination station

Station	As source	Share	As destination	Share	Source − destination
Bengaluru	701	10.4%	813	12.1%	−112
Chennai	1,296	19.2%	1,249	18.5%	+47
Delhi	1,295	19.2%	1,240	18.4%	+55
Gorakhpur	754	11.2%	730	10.8%	+24
Jaipur	694	10.3%	727	10.8%	−33
Kolkata	710	10.5%	731	10.8%	−21
Mumbai	1,288	19.1%	1,248	18.5%	+40

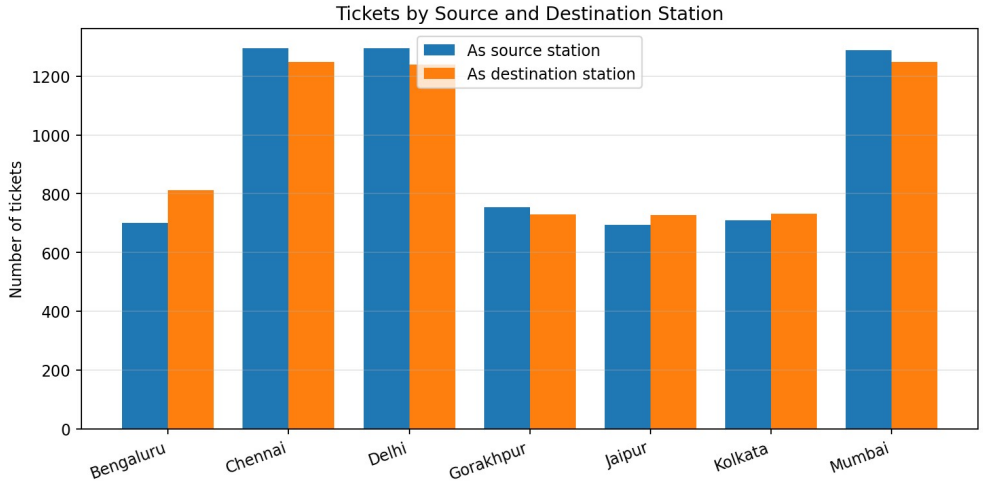


Figure 12: *Tickets by source and destination station.*

The last column of Table 24 shows the net direction of travel. Bengaluru has more tickets as a destination (813) than as a source (701), a difference of 112 tickets, which is the largest imbalance; Jaipur and Kolkata also have slightly more arrivals than departures, while Chennai, Delhi, Mumbai and Gorakhpur have slightly more departures. The Bengaluru imbalance comes mainly from two routes: Chennai to Bengaluru has 189 tickets against 142 in the opposite direction, and Delhi to Bengaluru has 188 tickets against 157. The differences sum to zero, as

they must. Because the dataset records only the tickets sold and not the return journeys, the imbalance shows the direction in which tickets were sold, not the net movement of passengers.

Table 25: Ten busiest directed routes

Rank	Route	Tickets	Share of all tickets
1	Mumbai - Chennai	337	5.0%
2	Delhi - Chennai	321	4.8%
3	Chennai - Delhi	320	4.7%
4	Mumbai - Delhi	317	4.7%
5	Chennai - Mumbai	316	4.7%
6	Delhi - Mumbai	314	4.7%
7	Chennai - Bengaluru	189	2.8%
8	Delhi - Bengaluru	188	2.8%
9	Delhi - Gorakhpur	171	2.5%
10	Gorakhpur - Chennai	165	2.4%

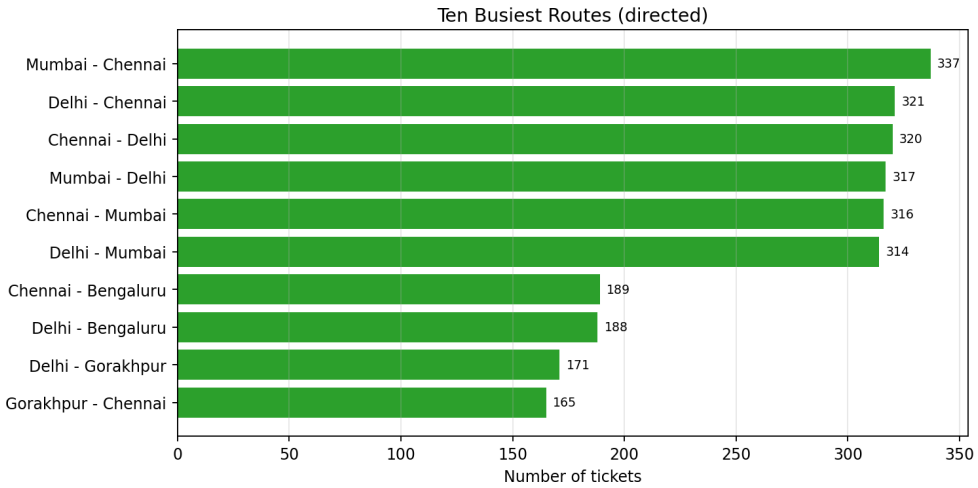


Figure 13: *The ten busiest directed routes.*

The ten busiest routes confirm a two-tier pattern. The first tier consists of the six routes among Chennai, Delhi and Mumbai with more than 310 tickets each. The second tier begins at 189 tickets, about 40% fewer than the weakest first-tier route (314 tickets), with routes from Chennai and Delhi to Bengaluru and from Delhi to Gorakhpur. Routes that connect only the smaller stations (Bengaluru, Gorakhpur, Jaipur and Kolkata) have the lowest counts, between 77 and 98 tickets. Since the dataset has no distance or train information, these results describe ticket demand only and do not indicate train capacity.

9. Key Findings

1. The cleaned dataset holds 6,738 tickets sold between January 2023 and September 2025 after removing 1,262 journeys with the same source and destination (15.8% of 8,000 records); no missing values or repeated rows were found.
2. Sleeper is the most booked coach (3,380 tickets, 50.2%), followed by General (1,965; 29.2%) and AC (1,393; 20.7%).
3. Ticket price depends on the coach class: mean prices are Rs. 349.82 (General), Rs. 897.84 (Sleeper) and Rs. 2,016.30 (AC), and the three price ranges do not overlap.
4. AC contributes 43.0% of the ticket revenue with 20.7% of the tickets; General contributes 10.5% of the revenue with 29.2% of the tickets.

5. The overall mean ticket price is Rs. 969.25 and the median Rs. 844; the mean is higher than the median because of the AC tickets.
6. Ticket price shows no visible relation to the source station, the passenger age or the payment mode; mean prices differ little between stations within a coach class.
7. Passenger age is spread almost evenly between about 5 and 80 years, and the age profile is similar for all four payment modes.
8. The Chennai, Delhi and Mumbai routes dominate demand: the six routes among them carry 28.6% of all tickets, and the three cities are the source of 57.6% of tickets.
9. Bengaluru has the largest imbalance between tickets sold as destination (813) and as source (701).
10. Monthly ticket revenue fluctuates between about Rs. 170,000 and Rs. 215,000 from January 2023 to September 2025, with no clear long-term trend.

10. Recommendations

The following recommendations are general suggestions based on the findings above; they are not results of the notebook.

1. **Prioritise the high-demand corridors.** Capacity planning and passenger services should give first attention to the Delhi–Mumbai–Chennai corridors, with the links from Chennai and Delhi to Bengaluru as the next priority.
2. **Protect AC availability.** AC coaches bring 43.0% of the revenue from 20.7% of the tickets, so changes in the coach mix should be checked against their effect on revenue, while Sleeper and General coaches carry most of the passengers.
3. **Standardise station coding.** Stations were recorded both as names and as codes, and some tickets had the same source and destination. Using one station code and validating the journey at entry would remove most of the cleaning work and the loss of 15.8% of the records.
4. **Plan services for all ages.** Passengers are spread evenly from childhood to old age, so facilities such as lower-berth allocation and assistance should be available on all major routes and not targeted at one age band.
5. **Record the journey date and train.** Adding the journey date, departure time and train number to the ticket data would allow real peak travel periods and train-wise load to be studied; the sale date shows only when the ticket was bought.

11. Limitations

- The dataset has no train number, journey date or time, distance or seat information. Sale_Date is the date of sale and not of travel, so peak travel periods cannot be identified.
- Ticket price depends almost only on the coach class and is spread uniformly inside each class; the price shows no relation to the route or the age. This suggests that the data may be simulated and not taken from real ticketing records; the source of the file should be confirmed.
- Journeys with identical source and destination (1,262 rows, 15.8%) were removed as invalid. This is an assumption; the rows may be a data-entry problem or may represent a different kind of ticket, which would change the route counts.
- Every record has a unique Ticket_ID, so the duplicate check could not detect repeated bookings.
- The currency of the ticket price is not stated; Indian Rupees are assumed.

- The notebook does not print the number of tickets per payment mode or per age group, the weekly mean price, or the exact monthly revenue after May 2023. Values from the box plot, histogram and monthly chart in this report are read from the charts and are approximate.
- The notebook computes no correlation coefficients or statistical tests, so relationships (for example between age and price) are described from charts only.
- The NumPy section of the notebook contains generic array demonstrations, which do not describe passenger travel.

Notebook issues noted

- The cell that prints df.shape, df.dtypes and df.describe() ended with NameError: name “df” is not defined, because it was run before the file was uploaded. Its results are therefore not available and are not used in this report; the row count of 8,000 is taken from the notebook title cell.
- The source of the dataset is a placeholder (“add dataset link here”).
- The notebook uploads the file with google.colab.files.upload() and therefore runs only in Google Colab unless that line is replaced by a file path.
- Some outputs (Section 6.1) were produced before cleaning, so station codes such as SBC and HWH appear in them.

12. Conclusion

Train ticket demand in this dataset depends mainly on the route and the coach class. A small number of routes among Chennai, Delhi and Mumbai carry more than a quarter of all tickets, Sleeper is the most booked coach class and AC is the main source of revenue. Ticket price is set by the coach class and shows no relation to the passenger age, the source station or the payment mode, and passengers of all ages from about 5 to 80 years travel. Monthly ticket revenue fluctuates without a clear trend. The data had few quality problems: inconsistent station codes and labels, and a sixth of the records with the same source and destination. The project shows that careful cleaning, feature engineering, group-wise comparison and simple visualizations turn a table of 6,738 tickets into practical information for railway planning: where passengers travel, in which class and what they pay.

13. Future Scope

- Add journey date, departure time, train number and distance to study real peak travel periods, load per train and price per kilometre.
- Analyse day-of-week and seasonal patterns of ticket sales and forecast monthly demand with a time-series model.
- Build a regression model to predict the ticket price from the coach type, route and age, and a classification model for the coach type.
- Count and compare the tickets per payment mode and per age group, and test the differences with statistical tests.
- Compute the correlation matrix of the numeric variables and test the apparent independence of price and age.
- Cluster stations by their incoming and outgoing demand to support network planning.
- Build a Streamlit or Dash dashboard so that non-technical users can explore the routes and prices.

References

1. The pandas development team. pandas documentation. https://pandas.pydata.org
2. McKinney, W. Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 2010.
3. Harris, C. R. et al. Array programming with NumPy. Nature 585, 2020. https://numpy.org
4. Hunter, J. D. Matplotlib documentation. https://matplotlib.org
5. Waskom, M. L. seaborn: statistical data visualization. Journal of Open Source Software 6(60), 3021, 2021. https://seaborn.pydata.org
6. Tukey, J. W. Exploratory Data Analysis. Addison-Wesley, 1977.
7. Add: original publisher, link and access date of train_ticket_sales_india.csv (the notebook contains only the placeholder “add dataset link here”).

Appendix A – Notebook (ipynb) File

The complete notebook edavproject112.ipynb is reproduced below with its code and outputs. The dataset is loaded in Google Colab with files.upload() followed by pd.read_csv. The cell that prints the shape, data types and summary statistics ended with a NameError and is reproduced as recorded. Plots appear as images; wide tables are reproduced in a small monospaced font.

Train Passenger Travel Pattern Analysis

Dataset: Train Ticket Sales India (train_ticket_sales_india.csv), 8000 ticket records, Jan 2023 to Sep 2025
Source: (add dataset link here)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files
```

```
up = files.upload()
df = pd.read_csv(list(up.keys())[0])
df.head()
```

```
Saving train_ticket_sales_india (1).csv to train_ticket_sales_india (1) (1).csv
   Ticket_ID   Sale_Date Source_Station Destination_Station  Passenger_Age  \
0          1  2023-01-01            SBC               Delhi             24
1          2  2023-01-01         Mumbai             Chennai             46
2          3  2023-01-01            SBC              Mumbai             76
3          4  2023-01-01            BCT                 HWH             38
4          5  2023-01-01          Delhi               Delhi             54

  Coach_Type  Ticket_Price Payment_Mode
0    Sleeper          1065         Cash
1    General           338         Cash
2    General           420          UPI
3    Sleeper           872          UPI
4         AC          1653          UPI
```

```
print(df.shape)
print(df.dtypes)
df.describe()
```

```
NameError: name 'df' is not defined
```

Pandas basics

```
prices = pd.Series(df['Ticket_Price'].head(5).values, index=['t1', 't2', 't3', 't4', 't5'])
print(prices)
coach = pd.DataFrame({'Coach_Type': ['General', 'Sleeper', 'AC'], 'Level': [1, 2, 3]})
coach
```

```
t1    1065
t2     338
t3     420
t4     872
t5    1653
dtype: int64
  Coach_Type  Level
0    General      1
1    Sleeper      2
2         AC      3
```

```
print(df.loc[0:4, ['Source_Station', 'Destination_Station', 'Ticket_Price']])
print(df.iloc[:5, :4])
df[df['Ticket_Price'] > df['Ticket_Price'].mean()].head()
```

```
  Source_Station Destination_Station  Ticket_Price
0            SBC               Delhi          1065
1         Mumbai             Chennai           338
2            SBC              Mumbai           420
3            BCT                 HWH           872
4          Delhi               Delhi          1653
   Ticket_ID   Sale_Date Source_Station Destination_Station
0          1  2023-01-01            SBC               Delhi
1          2  2023-01-01         Mumbai             Chennai
```

```
2          3  2023-01-01            SBC              Mumbai
3          4  2023-01-01            BCT                 HWH
4          5  2023-01-01          Delhi               Delhi
   Ticket_ID   Sale_Date Source_Station Destination_Station  Passenger_Age  \
0          1  2023-01-01            SBC               Delhi             24
4          5  2023-01-01          Delhi               Delhi             54
5          6  2023-01-01         Mumbai                 SBC             15
6          7  2023-01-01            HWH           Gorakhpur             47
8          9  2023-01-01            MAS              Mumbai             12

  Coach_Type  Ticket_Price Payment_Mode
0    Sleeper          1065         Cash
4         AC          1653          UPI
5         AC          1748         Cash
6         AC          1778          UPI
8    Sleeper          1113   NetBanking
```

Cleaning

```
print(df.isnull().sum())
df['Passenger_Age'] = df['Passenger_Age'].fillna(df['Passenger_Age'].median())
df = df.dropna(subset=['Ticket_Price'])
codes = {'BCT': 'Mumbai', 'MAS': 'Chennai', 'NDLS': 'Delhi', 'HWH': 'Kolkata', 'SBC': 'Bengaluru'}
df['Source_Station'] = df['Source_Station'].replace(codes)
df['Destination_Station'] = df['Destination_Station'].replace(codes)
df['Payment_Mode'] = df['Payment_Mode'].replace({'NetBanking': 'Net Banking'})
df['Sale_Date'] = pd.to_datetime(df['Sale_Date'])
df['Ticket_Price'] = df['Ticket_Price'].astype(float)
print(df.duplicated().sum())
df = df.drop_duplicates()
df = df[df['Source_Station'] != df['Destination_Station']]
df.shape
```

```
Ticket_ID             0
Sale_Date             0
Source_Station        0
Destination_Station    0
Passenger_Age         0
Coach_Type            0
Ticket_Price          0
Payment_Mode          0
dtype: int64
0
(6738, 8)
```

Transformation

```
df['Pay_Code'] = df['Payment_Mode'].map({'Cash': 0, 'UPI': 1, 'Card': 2, 'Net Banking': 3})
df['Route'] = df['Source_Station'] + ' - ' + df['Destination_Station']
df.rename(columns={'Ticket_Price': 'Price'}).head(3)
```

```
   Ticket_ID  Sale_Date Source_Station Destination_Station  Passenger_Age  \
0          1 2023-01-01      Bengaluru               Delhi             24
1          2 2023-01-01         Mumbai             Chennai             46
2          3 2023-01-01      Bengaluru              Mumbai             76

  Coach_Type   Price Payment_Mode  Pay_Code               Route
0    Sleeper  1065.0         Cash         0   Bengaluru - Delhi
1    General   338.0         Cash         0    Mumbai - Chennai
2    General   420.0          UPI         1  Bengaluru - Mumbai
```

```
df['Age_Group'] = pd.cut(df['Passenger_Age'], bins=[0, 18, 35, 60, 100], labels=['Child', 'Young', 'Adult',
'Senior'])
df['Price_Level'] = pd.qcut(df['Ticket_Price'], 3, labels=['Low', 'Medium', 'High'])
df['Price_Rank'] = df['Ticket_Price'].rank(ascending=False)
df.sort_values('Ticket_Price', ascending=False).head()
```

```
      Ticket_ID  Sale_Date Source_Station Destination_Station  Passenger_Age  \
7590       7591 2025-08-04      Gorakhpur               Delhi             32
4745       4746 2024-08-15      Gorakhpur              Jaipur             27
2421       2422 2023-10-30          Delhi              Jaipur             14
7801       7802 2025-08-31          Delhi              Mumbai             61
2118       2119 2023-09-22        Chennai           Gorakhpur             31
```

```
     Coach_Type  Ticket_Price Payment_Mode  Pay_Code               Route  \
7590         AC        2500.0         Cash         0    Gorakhpur - Delhi
4745         AC        2500.0  Net Banking         3   Gorakhpur - Jaipur
2421         AC        2500.0         Cash         0       Delhi - Jaipur
7801         AC        2499.0         Cash         0       Delhi - Mumbai
2118         AC        2499.0         Cash         0  Chennai - Gorakhpur

     Age_Group Price_Level  Price_Rank
7590     Young        High         2.0
4745     Young        High         2.0
2421     Child        High         2.0
7801    Senior        High         5.0
2118     Young        High         5.0
```

```
df.query("Age_Group == 'Senior' and Price_Level == 'High'").head()
```

```
    Ticket_ID  Sale_Date Source_Station Destination_Station  Passenger_Age  \
31         32 2023-01-04        Kolkata              Jaipur             71
36         37 2023-01-05         Mumbai               Delhi             69
38         39 2023-01-05         Mumbai             Chennai             68
42         43 2023-01-06         Jaipur             Chennai             77
46         47 2023-01-06      Gorakhpur              Mumbai             61

   Coach_Type  Ticket_Price Payment_Mode  Pay_Code               Route  \
31         AC        1860.0          UPI         1    Kolkata - Jaipur
36    Sleeper        1055.0  Net Banking         3      Mumbai - Delhi
38    Sleeper        1065.0          UPI         1    Mumbai - Chennai
42         AC        1972.0          UPI         1    Jaipur - Chennai
46    Sleeper        1121.0         Card         2  Gorakhpur - Mumbai

   Age_Group Price_Level  Price_Rank
31    Senior        High       920.5
36    Senior        High      2226.0
38    Senior        High      2156.0
42    Senior        High       772.5
46    Senior        High      1867.5
```

Indexing and merging

```
multi = df.set_index(['Source_Station', 'Destination_Station']).sort_index()
multi.loc['Delhi'].head()
```

```
                    Ticket_ID  Sale_Date  Passenger_Age Coach_Type  \
Destination_Station
Bengaluru                  12 2023-01-02             69    General
Bengaluru                  56 2023-01-07             68    General
Bengaluru                 100 2023-01-13             75         AC
Bengaluru                 137 2023-01-17             68    Sleeper
Bengaluru                 278 2023-02-04             34    Sleeper

                    Ticket_Price Payment_Mode  Pay_Code              Route  \
Destination_Station
Bengaluru                  329.0         Cash         0  Delhi - Bengaluru
Bengaluru                  232.0          UPI         1  Delhi - Bengaluru
Bengaluru                 2197.0         Cash         0  Delhi - Bengaluru
Bengaluru                  714.0         Card         2  Delhi - Bengaluru
Bengaluru                  916.0         Cash         0  Delhi - Bengaluru

                   Age_Group Price_Level  Price_Rank
Destination_Station
Bengaluru             Senior         Low      5882.5
Bengaluru             Senior         Low      6516.0
Bengaluru             Senior        High       447.0
Bengaluru             Senior      Medium      4082.5
Bengaluru              Young      Medium      2963.0
```

```
both = pd.concat([df.head(3), df.tail(3)])
merged = df.merge(coach, on='Coach_Type', how='left')
print(both.shape, merged.shape)
merged[['Coach_Type', 'Level']].head()
```

```
(6, 13) (6738, 14)
  Coach_Type  Level
```

```
0    Sleeper      2
1    General      1
2    General      1
3    Sleeper      2
4         AC      3
```

```
g = df.groupby(['Source_Station', 'Coach_Type'])['Ticket_Price'].mean().reset_index()
pv = g.pivot(index='Source_Station', columns='Coach_Type', values='Ticket_Price')
st = pv.stack()
print(pv.round(0))
print(st.head())
st.unstack().head()
```

```
Coach_Type          AC  General  Sleeper
Source_Station
Bengaluru       2005.0    346.0    896.0
Chennai         2052.0    346.0    909.0
Delhi           2013.0    349.0    899.0
Gorakhpur       1998.0    352.0    894.0
Jaipur          2014.0    357.0    894.0
Kolkata         2014.0    349.0    899.0
Mumbai          2003.0    352.0    891.0
Source_Station  Coach_Type
Bengaluru       AC            2005.417219
                General        345.950249
                Sleeper        895.873926
Chennai         AC            2052.236641
                General        346.183623
dtype: float64
Coach_Type               AC     General     Sleeper
Source_Station
Bengaluru       2005.417219  345.950249  895.873926
Chennai         2052.236641  346.183623  909.066561
Delhi           2012.886861  348.561308  898.848624
Gorakhpur       1997.617450  352.352941  894.453865
Jaipur          2014.465278  357.305000  893.837143
```

Groupby and time series

```
print(df.groupby('Coach_Type')['Ticket_Price'].agg(['count', 'mean', 'max']))
df.groupby(['Payment_Mode', 'Age_Group'], observed=True)['Ticket_Price'].mean().unstack()
```

```
            count         mean     max
Coach_Type
AC           1393  2016.301508  2500.0
General      1965   349.820865   500.0
Sleeper      3380   897.840237  1200.0
Age_Group           Child        Young        Adult      Senior
Payment_Mode
Card          1015.246057   946.654596  1011.511464  978.213983
Cash           956.168790  1003.047120   899.169231  965.963675
Net Banking    963.487288  1024.539286   943.840796  958.693151
UPI            922.061224   970.359140   987.613670  969.410646
```

```
ts = df.set_index('Sale_Date').sort_index()
monthly = ts['Ticket_Price'].resample('ME').sum()
weekly = ts['Ticket_Price'].resample('W').mean()
print(monthly.head())
ts.loc['2024-01':'2024-03'].head()
```

```
Sale_Date
2023-01-31    208642.0
2023-02-28    198804.0
2023-03-31    207156.0
2023-04-30    204980.0
2023-05-31    196739.0
Freq: ME, Name: Ticket_Price, dtype: float64
            Ticket_ID Source_Station Destination_Station  Passenger_Age  \
Sale_Date
2024-01-01       2927         Mumbai              Jaipur             20
2024-01-01       2928        Chennai              Jaipur             17
2024-01-01       2929          Delhi              Mumbai             15
2024-01-01       2930        Chennai              Mumbai              6
```

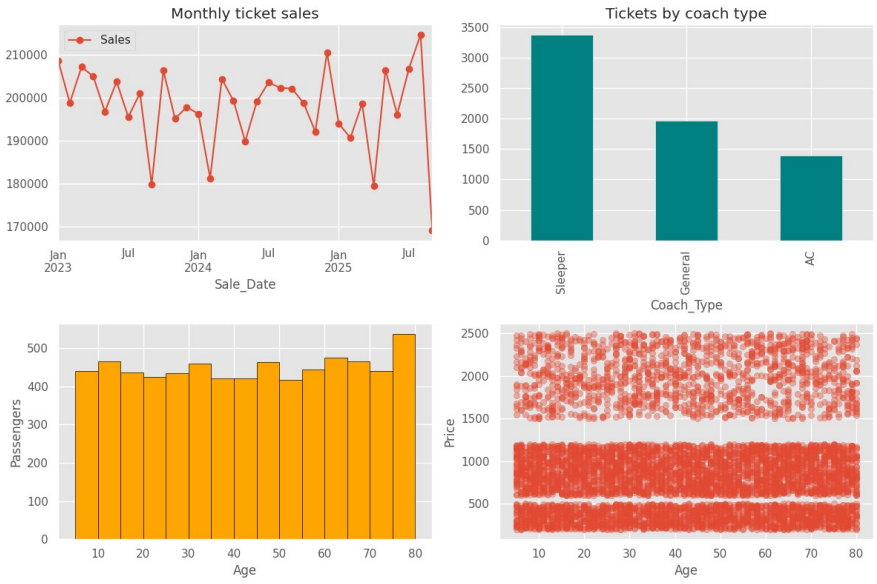
```
2024-01-01       2931         Jaipur               Delhi             23

           Coach_Type  Ticket_Price Payment_Mode  Pay_Code             Route  \
Sale_Date
2024-01-01    General         437.0         Card         2   Mumbai - Jaipur
2024-01-01    General         402.0          UPI         1  Chennai - Jaipur
2024-01-01    General         446.0  Net Banking         3    Delhi - Mumbai
2024-01-01    Sleeper        1192.0         Card         2  Chennai - Mumbai
2024-01-01    Sleeper         955.0  Net Banking         3    Jaipur - Delhi

           Age_Group Price_Level  Price_Rank
Sale_Date
2024-01-01     Young         Low      5195.5
2024-01-01     Child         Low      5422.5
2024-01-01     Child         Low      5135.5
2024-01-01     Child        High      1443.5
2024-01-01     Young      Medium      2762.0
```

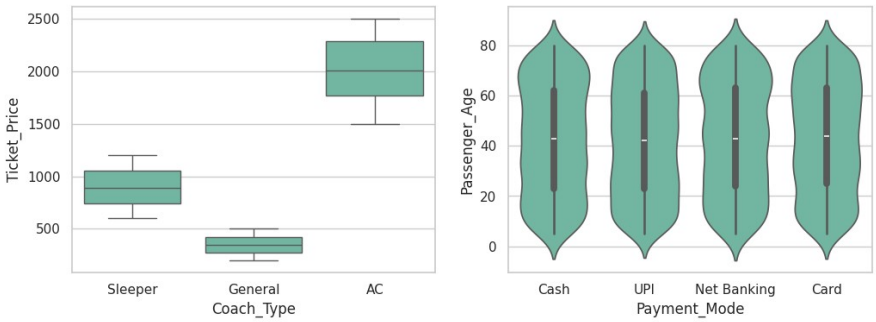
Matplotlib

```
plt.style.use('ggplot')
fig, ax = plt.subplots(2, 2, figsize=(12, 8))
monthly.plot(ax=ax[0, 0], marker='o', label='Sales')
ax[0, 0].set_title('Monthly ticket sales')
ax[0, 0].legend()
df['Coach_Type'].value_counts().plot(kind='bar', ax=ax[0, 1], color='teal')
ax[0, 1].set_title('Tickets by coach type')
ax[1, 0].hist(df['Passenger_Age'], bins=15, color='orange', edgecolor='black')
ax[1, 0].set_xlabel('Age')
ax[1, 0].set_ylabel('Passengers')
ax[1, 1].scatter(df['Passenger_Age'], df['Ticket_Price'], alpha=0.4)
ax[1, 1].set_xlabel('Age')
ax[1, 1].set_ylabel('Price')
plt.tight_layout()
plt.show()
```

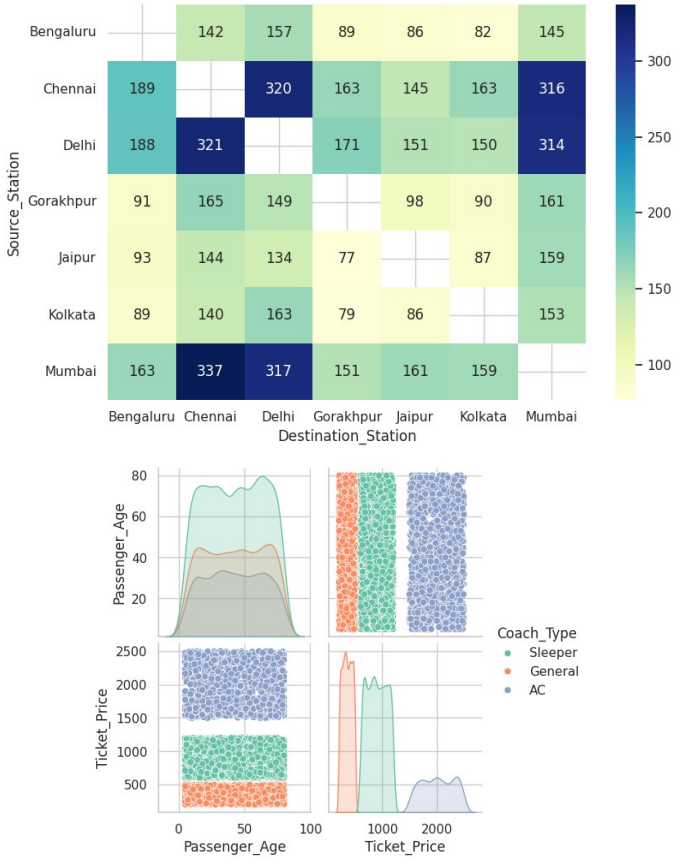


Seaborn

```
sns.set_theme(style='whitegrid', palette='Set2')
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
sns.boxplot(data=df, x='Coach_Type', y='Ticket_Price', ax=ax[0])
sns.violinplot(data=df, x='Payment_Mode', y='Passenger_Age', ax=ax[1])
plt.show()
```




```
routes = df.pivot_table(index='Source_Station', columns='Destination_Station', values='Ticket_ID',
aggfunc='count')
plt.figure(figsize=(9, 6))
sns.heatmap(routes, annot=True, fmt='.0f', cmap='YlGnBu')
plt.show()
sns.pairplot(df[['Passenger_Age', 'Ticket_Price', 'Coach_Type']], hue='Coach_Type')
plt.show()
```



NumPy

```
price = df['Ticket_Price'].to_numpy()
a1 = np.arange(1, 10)
a2 = a1.reshape(3, 3)
a3 = np.arange(24).reshape(2, 3, 4)
print(a2[1, :], a2[:2, 1:], a3[1, 2, 3])
print(a2 + np.array([10, 20, 30]))
print(a2 @ a2.T)
print(a2.T)
```

```
[4 5 6] [[2 3]
 [5 6]] 23
[[11 22 33]
 [14 25 36]
 [17 28 39]]
[[ 14  32  50]
```

```
 [ 32  77 122]
 [ 50 122 194]]
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

```
np.random.seed(1)
sample = np.random.choice(price, 50)
print(np.random.randint(1, 100, 5), np.random.rand(3))
print(price.mean(), np.median(price), price.std(), price.max())
print(a2.sum(axis=0), a2.mean(axis=1))
(price - price.mean()) / price.std()
```

```
[ 4 71 22 50 58] [0.1975509  0.28962964 0.14212014]
969.2499257940042 844.0 613.4104603486054 2500.0
[12 15 18] [2. 5. 8.]
array([ 0.15609462, -1.02908243, -0.89540359, ...,  0.286513  ,
       -0.88073152,  1.32496937])
```

Appendix B – How to Run

B.1 Running the notebook

1. Open edavproject112.ipynb in Google Colab (the notebook imports google.colab.files).
2. Run the cells from the top (Runtime → Run all). When the upload cell runs, select train_ticket_sales_india.csv; the notebook reads the first uploaded file, so the file name does not matter.
3. The libraries required are NumPy, pandas, Matplotlib and Seaborn, which are pre-installed in Google Colab. The monthly resampling uses the alias "ME", which needs pandas 2.2 or later.
4. All tables and charts are produced inside the notebook; the cells must be run in order, because the cell that prints df.shape and df.describe() needs the data frame df created by the upload cell.
5. To run the notebook outside Google Colab, delete the import of google.colab and replace the upload cell by df = pd.read_csv("train_ticket_sales_india.csv").